# CalibiAI

# Admin Portal

*Software Requirements Specification & Product Design Document*

| | |
|---|---|
| **Document Version** | 1.0 |
| **Status** | Implementation-Ready Draft |
| **Classification** | Internal — CalibiAI Team Only |
| **Primary Stack** | Next.js, Supabase, Amazon Bedrock |
| **Audience** | Frontend / Backend / AI / UI Teams |

# Table of Contents

# 1. Introduction & Objective

## 1.1 Overview

The CalibiAI Admin Portal is a standalone, internal-only web application that serves as the central operating system for the CalibiAI learning platform. It is architecturally and physically separate from the student-facing dashboard: it is deployed as its own application, uses its own routing domain (e.g. admin.calibiai.com), and has zero code-path overlap with student-facing UI. Students can never authenticate into, or be routed to, this application under any circumstance.

## 1.2 Goals

- Give CalibiAI staff a single place to monitor every student's learning journey in real time.
- Provide full content-management capability for roadmaps, assessments, quizzes, assignments, projects, research papers, articles, videos and the blog.
- Provide granular Role-Based Access Control so each staff function (Super Admin, Admin, Content Manager, Support Admin) only sees and can act on what it needs.
- Provide analytics and exportable reporting across the entire student base.
- Provide operational tooling: notifications, email templates, AI/Bedrock configuration, and platform settings.
- Be enterprise-grade in visual polish (Vercel / Linear / Notion / Supabase / Stripe inspired), fully responsive, accessible, and fast.

## 1.3 Non-Goals

- The Admin Portal will not expose any student-facing learning experience (no course-taking UI, no quiz-taking UI for students).
- The Admin Portal will not support public registration; accounts are provisioned exclusively by a Super Admin.

## 1.4 Technology Stack

| Layer | Technology | Notes |
|---|---|---|
| Frontend Framework | Next.js 14+ (App Router, TypeScript) | Server components for data-heavy admin tables |
| Styling | Tailwind CSS + shadcn/ui | Design tokens defined in Section 17 |
| State/Data | TanStack Query + Zustand | Server cache + light client state |
| Auth | Supabase Auth (email/password) | JWT with custom `role` claim |
| Database | Supabase Postgres | Row Level Security enforced on every table |
| Storage | Supabase Storage | Buckets for blog images, avatars, exports |
| AI Layer | Amazon Bedrock (Claude models) | Talent scoring, dropout prediction, recommendations |
| Email | Supabase Edge Function + Resend/SES | Transactional + campaign email |

| Layer | Technology | Notes |
|---|---|---|
| Charts | Recharts | All analytics visualizations |
| Hosting | Vercel (frontend) + Supabase Cloud (backend) | Separate project from student dashboard |

# 2. Authentication & Login

## 2.1 Purpose

Provide a secure, minimal login surface for pre-provisioned CalibiAI staff accounts only. There is no self-service signup anywhere in the Admin Portal.

## 2.2 Login Screen — UI Specification

- Centered card (max-width 400px) on a white background with a subtle green radial gradient in the top-left corner.
- CalibiAI logo top-center, followed by heading 'Sign in to Admin Portal'.
- Fields: Email (type=email, autofocus), Password (type=password, with show/hide toggle icon).
- Primary button 'Sign In' — full width, green (#12B76A), 44px height, disabled state while submitting with spinner.
- 'Forgot password?' link right-aligned under the password field.
- 'Remember me' checkbox left-aligned under the password field.
- Footer micro-copy: 'Authorized CalibiAI personnel only. Unauthorized access attempts are logged.'

## 2.3 Authentication Flow

1. User submits email + password from the login form.
2. Frontend calls `supabase.auth.signInWithPassword({ email, password })`.
3. On success, the client fetches the row from `admin_profiles` matching `auth.uid()`.
4. If no matching row exists, or `admin_profiles.is_active = false`, the session is immediately signed out and the user is redirected to `/access-denied`.
5. If a matching active row exists, the `role` and `permissions` are loaded into a Zustand auth store and the user is redirected to `/dashboard`.
6. Every login attempt (success or failure) is written to the `admin_login_logs` table via an Edge Function trigger for audit purposes.

## 2.4 Forgot Password

- Uses `supabase.auth.resetPasswordForEmail(email, { redirectTo: '/reset-password' })`.
- Generic confirmation message is shown regardless of whether the email exists, to avoid account enumeration: 'If that email is registered, a reset link has been sent.'
- Reset link expires in 60 minutes (Supabase default is configurable in Auth settings).

## 2.5 Remember Login & Session Management

- 'Remember me' checked → Supabase session persisted in `localStorage` with refresh-token rotation (default Supabase behavior); unchecked → session stored in memory only, cleared on tab close via `sessionStorage`-backed custom storage adapter.
- Access token lifetime: 1 hour. Refresh token rotates automatically via `supabase-js` auto-refresh.
- Idle timeout: after 30 minutes of no interaction (mouse/keyboard/route change), the client shows a 'Session expiring' modal with a 60-second countdown, then force-signs-out.
- Absolute session ceiling: 12 hours, after which re-authentication is required regardless of activity.

## 2.6 Role Validation & Access Denied

Role validation happens twice: (1) client-side, to route users to the correct default landing page and hide unauthorized navigation items, and (2) server-side, via Postgres Row Level Security policies and Edge Function guards, which are the actual source of truth. Client-side checks are a UX convenience only and are never trusted for authorization decisions.

> **Security Note**
> Never rely on hiding a UI element as the sole access control. Every table read/write is additionally gated by an RLS policy keyed off the JWT's `role` claim, and every Edge Function re-validates the caller's role from `admin_profiles` before executing privileged logic.

## 2.7 Backend Services

- Supabase Auth (email/password provider only; magic link and OAuth disabled for this project).
- Postgres trigger `on_auth_user_created` — blocks self-registration by rejecting any `auth.users` insert that does not originate from the `provision_admin` Edge Function (checked via a service-role-only insert path).
- Edge Function `provision-admin` — used exclusively by Super Admins to create new staff accounts (see Section 8, Settings → Admins).

## 2.8 Supabase Tables

- `admin_profiles` — id (uuid, FK auth.users), full_name, email, role, permissions (jsonb), is_active, avatar_url, created_at, created_by, last_login_at.
- `admin_login_logs` — id, admin_id (nullable, for failed unknown-email attempts), email_attempted, success (bool), ip_address, user_agent, failure_reason, created_at.

## 2.9 API Endpoints

| Method | Endpoint | Description |
|---|---|---|
| POST | /auth/v1/token?grant_type=password | Supabase-managed sign-in |
| POST | /auth/v1/recover | Supabase-managed password reset request |
| POST | /functions/v1/provision-admin | Super Admin creates a new staff account |
| POST | /functions/v1/log-login-attempt | Writes to admin_login_logs (called from client on every attempt) |
| GET | /rest/v1/admin_profiles?id=eq.{uid} | Fetch current admin's profile & role after login |

## 2.10 Validation Rules

- Email must match `^[^\s@]+@[^\s@]+\.[^\s@]+$` and, additionally, must exist as an active row in `admin_profiles` — unknown emails fail with a generic 'Invalid email or password' message (no enumeration).
- Password minimum 10 characters, at least 1 uppercase, 1 number, 1 symbol (enforced at account-creation time by the `provision-admin` function; Supabase Auth password policy mirrors this).
- Rate limit: 5 failed attempts per email per 15 minutes → account temporarily locked (see Section 15 Security).

## 2.11 Edge Cases

- Admin deactivated mid-session → next API call returns 401; client force-signs-out with message 'Your access has been revoked. Contact a Super Admin.'
- Admin role changed mid-session → JWT custom claim is stale for up to the access-token lifetime (1 hr); client re-fetches `admin_profiles` on every route change to mitigate, and Edge Functions always re-check the DB row rather than trusting the JWT claim alone.
- Two staff members editing the same content record concurrently → optimistic locking via an `updated_at` version check (see Section 6 & 7).

## 2.12 Loading & Error States

- Login button shows inline spinner + disabled state; form fields disabled during submission.
- Network failure → toast: 'Unable to reach CalibiAI servers. Check your connection and try again.'
- Invalid credentials → inline error under the password field, form fields re-enabled, password field cleared.

## 2.13 Future Scalability

- Add SSO (Google Workspace / Okta) for staff once headcount grows past ~25.
- Add mandatory 2FA (TOTP) for Super Admin and Admin roles.
- Add device/session management screen so admins can view and revoke active sessions.

# 3. Admin Roles & Permissions (RBAC)

## 3.1 Purpose

Enforce least-privilege access across four distinct staff roles. Permissions are stored both as a coarse `role` enum (for fast RLS checks) and a fine-grained `permissions` JSONB column (for UI feature-flagging and future custom roles).

## 3.2 Role Matrix

| Capability | Super Admin | Admin | Content Manager | Support Admin |
|---|---|---|---|---|
| View dashboard & analytics | Yes | Yes | Limited (content stats only) | Limited (student activity only) |
| Manage students (edit/deactivate) | Yes | Yes | No | No — search/view/reset only |
| View student personal data (PII) | Yes | Yes | No | Yes |
| Publish / edit blogs | Yes | Yes | Yes | No |
| Manage roadmaps, quizzes, assignments | Yes | Yes | No | No |
| Manage research papers / articles / videos | Yes | Yes | Yes | No |
| Delete any record | Yes | Content only, not students/admins | Own content only | No |
| Manage admins & roles | Yes | No | No | No |
| Delete a Super Admin | No (protected) | No | No | No |
| Export reports (CSV/Excel/PDF) | Yes | Yes | No | View-only, no export |
| Resend onboarding / reset onboarding | Yes | Yes | No | Yes |
| Configure AI / Bedrock / API keys | Yes | No | No | No |
| Send notifications | Yes | Yes | Blog-related only | No |

## 3.3 Permission Model — Implementation

`admin_profiles.role` is one of: `super_admin`, `admin`, `content_manager`, `support_admin`. The `permissions` JSONB column stores fine-grained overrides for future custom roles, structured as:

```
{
```

```
  "students": { "view": true, "edit": true, "delete": false },
  "blogs": { "view": true, "create": true, "publish": true, "delete": false },
  "content": { "roadmaps": "write", "assessments": "write", "videos": "read" },
  "settings": { "manage_admins": false, "manage_ai": false },
  "export": true
}
```

At request time, RLS policies check `role` for coarse gating and Edge Functions additionally check `permissions` for fine-grained gating on write operations.

### 3.4 Row Level Security Pattern

```
-- Example: students table SELECT policy
create policy "students_select_by_role" on students
for select using (
  auth.jwt() ->> 'role' in ('super_admin','admin','support_admin')
);

-- Example: students table UPDATE policy (support_admin excluded)
create policy "students_update_by_role" on students
for update using (
  auth.jwt() ->> 'role' in ('super_admin','admin')
);
```

### 3.5 Supabase Tables

- `admin_roles` — role (pk text), label, description, is_system_role (bool) — seeds the four built-in roles and allows future custom roles.
- `admin_profiles.role` — FK to `admin_roles.role`.
- `admin_permission_audit` — logs every permission/role change: id, target_admin_id, changed_by, old_role, new_role, old_permissions, new_permissions, created_at.

### 3.6 API Endpoints

| Method | Endpoint | Description |
|---|---|---|
| GET | /rest/v1/admin_roles | List all roles and their default permission templates |
| PATCH | /functions/v1/admin/{id}/role | Super Admin changes a staff member's role |
| PATCH | /functions/v1/admin/{id}/permissions | Super Admin overrides fine-grained permissions |
| GET | /rest/v1/admin_permission_audit?target_admin_id=eq.{id} | Permission change history for one admin |

## 3.7 Edge Cases

- Attempt to delete/downgrade the last remaining Super Admin → blocked server-side with 409 Conflict: 'At least one Super Admin must remain.'
- Admin attempts to delete a Super Admin record → 403 Forbidden regardless of caller's own role (hard-coded server-side rule, not just RLS).
- Content Manager attempts to open a student profile URL directly → server returns 403; UI shows an Access Denied illustration with a 'Return to Dashboard' button.

## 3.8 Future Scalability

- Fully custom, admin-defined roles built from the `permissions` JSONB schema via a UI role-builder.
- Scoped permissions (e.g. Content Manager limited to specific blog categories only).

# 4. Admin Dashboard

## 4.1 Purpose

The landing screen after login. Gives every role an at-a-glance, role-filtered pulse on the platform: growth, engagement, learning outcomes, and recent activity.

## 4.2 Layout

- Persistent left sidebar (240px, collapsible to 64px icon rail) — logo, primary nav grouped into Overview / Students / Content / Blog / Analytics / Notifications / Settings, with active-item green left-border indicator.
- Top bar — search (Cmd+K command palette), notification bell with unread badge, admin avatar menu (Profile, Theme, Logout).
- Main canvas — 12-column responsive grid, 24px gutter, max-width 1440px, centered.
- Row 1: statistics cards grid (auto-fit, min 220px per card).
- Row 2: two-column — Recent Activity Feed (left, 60%) and Notifications panel (right, 40%).
- Row 3: quick-glance charts — DAU/WAU/MAU trend line and Roadmap completion funnel.

## 4.3 Statistics Cards

Each stat card: label (grey, 13px), big number (32px bold), small delta badge (green ▲ / red ▼ vs previous period), and a 7-day sparkline. Cards shown are filtered by role (Content Manager sees content/blog stats only; Support Admin sees activity stats only).

- Total Students, Active Students Today, New Students (7d)
- Completed Assessments, Average Assessment Score, Average Talent Score
- Roadmaps Assigned, Roadmap Completion Rate
- Daily Active Users, Weekly Active Users, Monthly Active Users
- Projects Submitted, Assignments Completed
- Total Blog Views, Community Users, GitHub Connected Users

## 4.4 Recent Activity Feed

Reverse-chronological, virtualized list (react-virtual) pulling from a unified `admin_activity_feed` view that unions recent registrations, blog posts, assessments, projects, and system errors. Each row: icon (colored by event type), description, relative timestamp, and a link to the relevant record.

## 4.5 Backend Services

- Materialized view `dashboard_stats_daily`, refreshed hourly via `pg_cron`, pre-aggregates counts to keep the dashboard sub-200ms.
- Edge Function `get-dashboard-summary` composes the stat cards payload, applying role-based field filtering before returning to the client (defense in depth alongside RLS).
- `admin_activity_feed` is a Postgres view unioning `students`, `blog_posts`, `assessment_attempts`, `projects`, and `system_error_logs`, ordered by `created_at desc`, limited server-side to the most recent 200 rows.

## 4.6 Supabase Tables

- `dashboard_stats_daily` (materialized view) — metric_date, total_students, active_students, new_students, avg_assessment_score, avg_talent_score, dau, wau, mau, projects_submitted, assignments_completed, blog_views, community_users, github_connected.
- `system_error_logs` — id, source, message, stack_trace, severity, resolved (bool), created_at.

## 4.7 API Endpoints

| Method | Endpoint | Description |
|---|---|---|
| GET | /functions/v1/get-dashboard-summary | Role-filtered stat cards payload |
| GET | /rest/v1/admin_activity_feed?limit=50 | Recent activity feed, paginated via `offset` |
| GET | /rest/v1/dashboard_stats_daily?order=metric_date.desc&limit=30 | 30-day trend data for sparklines/charts |

## 4.8 Loading & Error States

- Stat cards render skeleton shimmer blocks (fixed height, matching final layout) while loading — no layout shift.
- Activity feed shows 6 skeleton rows, then an empty state illustration + 'No recent activity yet' if genuinely empty.
- If `get-dashboard-summary` fails, cards show a muted dash '—' with a small retry icon rather than blocking the whole page.

## 4.9 Future Scalability

- Per-admin customizable dashboard (drag-to-reorder cards, saved layout in `admin_profiles.dashboard_layout` jsonb).
- Real-time updates via Supabase Realtime channel instead of hourly materialized-view refresh, for teams that need live numbers.

# 5. Student Management Module

## 5.1 Overview

A dense, sortable, filterable data table listing every student on the platform, built for admins who need to triage hundreds or thousands of learners quickly. Not visible to Content Manager.

## 5.2 Table Columns

- Profile Picture (avatar, 32px circular)
- Name (bold, links to profile)
- Email
- Phone
- College
- Branch
- Graduation Year
- Role Selected (e.g. Frontend, ML, DevOps)
- Assessment Score (badge, color-coded: red <40, amber 40-70, green >70)
- Talent Score (0-1000, same color banding)
- Current Roadmap
- Current Day (e.g. Day 14/90)
- Progress (inline progress bar, %)
- GitHub Connected (icon: check/cross)
- Last Login (relative time, red text if >30 days)
- Account Status (Active / Inactive / Suspended pill)

## 5.3 Search, Filter, Sort, Pagination

- Global search box (debounced 300ms) searching name, email, phone via a Postgres trigram index (`pg_trgm`) for fast fuzzy match.
- Filter drawer (right-side sheet) with: College (multi-select, typeahead), Branch, Role, Assessment Score (range slider), Talent Score (range slider), Roadmap (select), Current Day (range), Activity (Active in: 7d/30d/90d/Never), Account Status, Date Joined (date range), GitHub Connected (yes/no/any).
- Active filters render as removable chips above the table; 'Clear all' resets to default.
- Column-header click toggles sort (asc/desc/none), with a small caret icon; sort state persisted in the URL query string so filtered/sorted views are shareable and bookmarkable.
- Server-side pagination: 25/50/100 rows per page selector; page-jump input for large result sets; total count shown ('Showing 1-50 of 4,312 students').
- Row checkbox + header 'select all on page' / 'select all matching filter' (with a clear indicator of how many are selected when the latter is used).

## 5.4 Bulk Actions

- Deactivate selected

- Reactivate selected
- Resend onboarding email to selected
- Reset onboarding progress for selected (Support Admin + above)
- Export selected (see 5.5)
- Assign roadmap to selected (Admin + above)

## 5.5 Export

Every view of the table — filtered, sorted, or a manual selection — is exportable. Export runs as an async background job for result sets over 2,000 rows to avoid blocking the UI.

- Formats: CSV, Excel (.xlsx, formatted with header styling), PDF (paginated, portrait, CalibiAI letterhead).
- Scope options at export time: 'All students', 'Filtered results', 'Selected rows only'.
- Saved filter presets act as one-click export shortcuts, e.g.: All Students; Only Active Students; Only IIT Colleges; Only Pune Colleges; Talent Score above 700; Assessment Score below 40; Not logged in for 30+ days; Completed Roadmap; No GitHub Connected.
- Large exports (>2,000 rows) are queued via an Edge Function, written to a private `exports` Storage bucket, and the admin is notified in-app + by email with a signed, expiring (24h) download link.

## 5.6 Backend Services

- Edge Function `list-students` — accepts filter/sort/pagination params, builds a parameterized query (never string-concatenated) against `students`, returns rows + total count.
- Edge Function `export-students` — validates scope, either streams a small CSV directly or enqueues a job row in `export_jobs` processed by a background worker for large sets.
- Edge Function `bulk-student-action` — validates the caller's role/permissions, applies the action inside a single transaction, and writes one `admin_activity_logs` row per affected student for auditability.

## 5.7 Supabase Tables

- `students` — id, auth_user_id, full_name, email, phone, college, branch, graduation_year, role_selected, avatar_url, github_username, github_connected, account_status, current_roadmap_id, current_day, progress_percent, assessment_score, talent_score, created_at, last_login_at.
- `export_jobs` — id, requested_by, scope, filters (jsonb), format, status (queued/processing/complete/failed), file_url, row_count, created_at, completed_at.
- Index: `idx_students_search` (GIN, trigram, on `full_name || email || phone`); `idx_students_filters` (btree, on college, branch, account_status, last_login_at).

## 5.8 API Endpoints

| Method | Endpoint | Description |
|---|---|---|
| GET | /functions/v1/list-students | Filtered, sorted, paginated student list |
| POST | /functions/v1/export-students | Trigger CSV/Excel/PDF export (sync or queued) |
| GET | /rest/v1/export_jobs?requested_by=eq.{id} | Poll export job status |
| POST | /functions/v1/bulk-student-action | Deactivate / reactivate / resend / reset-onboarding in bulk |

| Method | Endpoint | Description |
|---|---|---|
| PATCH | /rest/v1/students?id=eq.{id} | Edit a single student's editable fields |

## 5.9 Validation Rules

- Filter range values validated server-side (e.g. talent score 0-1000) and clamped rather than erroring, to keep the UI forgiving.
- Bulk action payload capped at 5,000 student IDs per request; larger sets must go through 'select all matching filter' which is handled as a filter-based server operation, not an ID array.
- Export format must be one of `csv|xlsx|pdf`; invalid values return 422.

## 5.10 Edge Cases

- 'Select all matching filter' selection while filters change → selection is invalidated and the UI prompts the admin to re-select rather than silently acting on a stale filter.
- Student deletes their own GitHub connection after being included in an export → export reflects data as of generation time; each export file includes a 'Generated at' timestamp.
- Two admins export the same filter simultaneously → each gets an independent `export_jobs` row; no locking required since exports are read-only.

## 5.11 Loading & Error States

- Table shows 10 skeleton rows on first load and a thin top progress bar on subsequent filter/sort/page changes (no full-table skeleton flash for pagination, to reduce visual noise).
- Empty filtered result → illustration + 'No students match these filters' + 'Clear filters' button.
- Export failure → toast with 'Export failed — Retry' action; failure reason logged in `export_jobs.status = failed` with an error message surfaced on hover.

## 5.12 Future Scalability

- Saved custom filter presets per-admin, shareable across the team.
- Cursor-based (keyset) pagination once table size passes ~500k rows for consistent performance.
- Column customization (show/hide/reorder) persisted per-admin.

# 6. Student Profile Page

## 6.1 Purpose

A 360-degree view of a single student, opened from the Student Management table or search. Organized as a tabbed layout so admins can drill into exactly the dimension they need without a single overwhelming page.

## 6.2 Layout

- Header band: avatar, name, email/phone, college/branch/year, account status pill, quick actions (Deactivate, Reset Onboarding, Resend Email, Impersonate-View — read-only preview of the student dashboard, Super Admin/Admin only).
- Tab bar: Overview · Learning Progress · Assessments · Skill Graph · Roadmap & Missions · Quizzes & Assignments · Projects · GitHub Analysis · Talent Score Timeline · Weekly Reports · Activity & Login History.

### Overview Tab

Summary cards: current roadmap + day, progress %, talent score with trend arrow, assessment score, GitHub connection status, last login. A condensed activity timeline (last 10 events) below.

### Learning Progress Tab

Day-by-day roadmap progress as a horizontal stepper (completed / current / locked), with time-spent-per-day bar chart.

### Assessment Results Tab

List of all assessment attempts: date, score, duration, pass/fail, breakdown by topic/skill area (radar chart), and a link to view the full attempt (question-by-question) in a read-only viewer.

### Skill Graph Tab

Radar/spider chart of skill categories (e.g. DSA, System Design, Frontend, Backend, AI/ML) scored 0-100, derived from assessment + project + quiz performance, computed by the AI scoring service.

### Roadmap & Daily Missions Tab

The assigned roadmap's full day list with per-day mission completion status, timestamps, and admin override capability to mark a day complete/incomplete (Admin+ only, logged in audit trail).

### Quiz Attempts & Assignments Tab

Tables of every quiz attempt (score, attempt number, timestamp) and every assignment submission (status: submitted/graded/late, grade, submitted file/link, grader's feedback).

### Projects Tab

Card grid of submitted projects: title, description, repo link, submission date, review status, reviewer feedback, and an AI-generated project-quality summary.

### GitHub Analysis Tab

If connected: commit frequency heatmap, repo count, languages used, contribution graph, and an AI-generated summary of coding activity/quality signals.

### Talent Score Timeline Tab

Line chart of talent score over time, annotated with events that caused notable jumps (assessment passed, project reviewed, roadmap milestone).

### Weekly Reports Tab

Auto-generated weekly summaries (AI-authored) of the student's progress, strengths, and risk flags (e.g. inactivity, declining scores), viewable and downloadable as PDF.

### Activity Timeline & Login History Tab

Unified, filterable, infinite-scroll log of every tracked student event (logins, submissions, roadmap progress, GitHub sync) with IP/device metadata on login rows.

## 6.3 Backend Services

- Edge Function `get-student-profile` — a single composed call that fans out to the relevant tables/views per tab (lazy-loaded per tab click to avoid over-fetching on initial page load).
- AI scoring pipeline (Bedrock) recomputes skill-graph and weekly reports nightly via a scheduled job; on-demand recompute button available to Admin+.

## 6.4 Supabase Tables

- `assessment_attempts`, `quiz_attempts`, `assignments`, `projects`, `roadmap_progress`, `daily_missions`, `github_analysis`, `talent_score_history`, `weekly_reports`, `student_activity_log`, `admin_login_logs` (for admin-side login events; student logins live in `student_login_logs`).

## 6.5 API Endpoints

| Method | Endpoint | Description |
|---|---|---|
| GET | /functions/v1/get-student-profile?id={id}&tab=overview | Lazy-loaded per-tab profile data |
| PATCH | /functions/v1/students/{id}/override-day | Admin overrides a roadmap day's completion status |
| POST | /functions/v1/students/{id}/recompute-scores | Force-recompute talent/skill scores |
| GET | /functions/v1/students/{id}/weekly-report/{week}/pdf | Download a weekly report as PDF |

## 6.6 Edge Cases

- Student never connected GitHub → GitHub Analysis tab shows an explanatory empty state, not an error.
- Assessment retaken multiple times → all attempts retained; 'best score' and 'latest score' both clearly labeled to avoid ambiguity.
- Admin override of a roadmap day is always logged with before/after state in `admin_activity_logs` and visible in the student's Activity Timeline tagged 'Modified by Admin'.

## 6.7 Future Scalability

- Add a 'Compare Students' side-by-side view.
- Add AI-generated intervention recommendations directly on the profile (ties into Future Features Section 16).

# 7. Blog Management (CMS)

## 7.1 Purpose

A full content-management system for CalibiAI's blog, used by Content Manager, Admin, and Super Admin. Publishing a post makes it immediately live in the student dashboard's Blog section and triggers a student notification.

## 7.2 Blog Post Fields

| Field | Type | Notes |
|---|---|---|
| Title | text, required | 3-150 chars |
| Slug | text, required, unique | Auto-generated from title, editable, kebab-case enforced |
| Cover Image | image upload | Required to publish; 1200x630 recommended, max 5MB |
| Short Description | textarea, required | Max 200 chars, used in cards & meta description fallback |
| Content | rich text (Markdown-backed) | See 7.3 editor spec |
| Author | select from admin_profiles | Defaults to current admin |
| Category | select, required | AI, LLMs, RAG, Career, Automation, News, Research, Prompt Engineering |
| Tags | multi-select / freeform | Max 8 tags |
| Reading Time | auto-computed | words / 200wpm, rounded up; manually overridable |
| SEO Title | text, optional | Falls back to Title if empty; max 60 chars |
| SEO Description | text, optional | Falls back to Short Description; max 160 chars |
| Featured Image | image upload, optional | Used for social share cards (OG image) |
| Publish Date | datetime | Can be future-dated for scheduling |
| Status | enum | Draft, Published, Archived |

## 7.3 Rich Text Editor

- Built on Tiptap (ProseMirror), storing content as Markdown in `blog_posts.content_markdown` plus a rendered `content_html` cache column for fast student-dashboard reads.
- Toolbar: headings (H2-H4), bold/italic/strike, ordered/unordered lists, blockquote, code block (syntax-highlighted), inline code, link, image insert (uploads to Storage), table, horizontal rule, undo/redo.
- Image upload drag-and-drop directly into the editor body, auto-optimized (WebP conversion, max 1600px width) via an Edge Function on upload.
- Full Markdown shortcuts supported (e.g. typing '## ' converts to H2) for power users.
- Autosave draft every 15 seconds and on blur; manual 'Save Draft' button also available; unsaved-changes browser warning on navigation.

## 7.4 Workflow: Draft → Preview → Publish

7. Author creates a post; status defaults to Draft. Draft posts are never visible to students.
8. 'Preview' opens a read-only render in a new tab styled exactly as the student dashboard will render it.
9. 'Publish' either publishes immediately (status → Published, `published_at = now()`) or, if a future Publish Date is set, schedules it (status stays Draft internally with `scheduled_at` set; a `pg_cron` job flips status to Published at the scheduled time).
10. On transition to Published, a Postgres trigger fires an Edge Function that (a) invalidates the student dashboard's blog cache and (b) enqueues an in-app + push notification to all students (see Section 9 Notifications).
11. 'Archive' hides a previously published post from students without deleting it; it remains editable and can be re-published.

## 7.5 Version History

Every save (draft autosave excluded — only explicit saves and status transitions) writes an immutable snapshot to `blog_post_versions`. The Version History panel lists snapshots with author + timestamp, a diff view (added/removed text highlighted), and a 'Restore this version' action that creates a new version rather than destructively overwriting.

## 7.6 Backend Services

- Edge Function `publish-blog-post` — validates required fields (cover image, category), sets status/timestamps, fires the notification fan-out.
- Edge Function `generate-blog-slug` — slugifies title, appends `-2`, `-3` etc. on collision.
- `pg_cron` job `publish-scheduled-posts` — runs every minute, publishes any post whose `scheduled_at <= now()` and status is still pending-scheduled.
- Image optimization Edge Function, triggered on Storage upload webhook.

## 7.7 Supabase Tables

- `blog_posts` — id, title, slug, cover_image_url, short_description, content_markdown, content_html, author_id, category, tags (text[]), reading_time_minutes, seo_title, seo_description, featured_image_url, status, published_at, scheduled_at, created_at, updated_at, updated_by.
- `blog_post_versions` — id, blog_post_id, snapshot (jsonb), author_id, created_at.
- `blog_categories` — slug (pk), label, description.
- Storage bucket `blog-images` (public-read, admin-write via signed upload).

## 7.8 API Endpoints

| Method | Endpoint | Description |
|---|---|---|
| GET | /rest/v1/blog_posts?order=updated_at.desc | List posts (CMS table view) |
| POST | /rest/v1/blog_posts | Create draft |
| PATCH | /rest/v1/blog_posts?id=eq.{id} | Update / autosave |
| POST | /functions/v1/publish-blog-post | Publish or schedule |
| PATCH | /functions/v1/blog-posts/{id}/archive | Archive a published post |
| DELETE | /rest/v1/blog_posts?id=eq.{id} | Hard delete (Super Admin only, drafts only) |

| Method | Endpoint | Description |
|---|---|---|
| GET | /rest/v1/blog_post_versions?blog_post_id=eq.{id} | Version history list |
| POST | /functions/v1/blog-posts/{id}/restore-version | Restore a prior version as new latest |

## 7.9 Validation Rules

- Cannot transition to Published without: title, slug, cover image, short description, content, category.
- Slug must be unique across all posts regardless of status; validated on blur with a live-availability check.
- Tags capped at 8; category must be one of the fixed enum values.

## 7.10 Edge Cases

- Two admins edit the same draft simultaneously → last-write-wins at the field level is avoided; instead an `updated_at` version check on save returns 409 with a 'This post was updated by {name} — reload to see changes' prompt.
- Scheduled post's scheduled time is edited to the past → publishes on the next `pg_cron` tick (within 1 minute) rather than erroring.
- Post archived while a student has it open → student dashboard shows a 'This article is no longer available' state rather than a broken page.

## 7.11 Loading & Error States

- Editor shows a skeleton toolbar + content placeholder while loading an existing draft.
- Autosave failures show a small non-blocking 'Draft not saved — retrying' indicator near the save status label, never a modal.
- Image upload failure → inline error within the editor at the insertion point with a retry button.

## 7.12 Future Scalability

- Multi-author collaborative editing (presence cursors) via Supabase Realtime + Yjs.
- AI-assisted drafting and SEO suggestions using Bedrock, surfaced as inline editor suggestions.
- Comment/approval workflow (Content Manager submits, Admin approves) before publish.

# 8. Content Management

## 8.1 Purpose

A unified hub for managing every non-blog learning asset: Roadmaps, Assessments, Quizzes, Assignments, Projects, Research Papers, Articles, and Videos. Each content type shares a common CRUD/versioning/preview/archive pattern, exposed as tabs within one Content Management screen so admins never have to jump between disconnected tools.

## 8.2 Shared Pattern Across All Content Types

- List view: searchable, filterable (by status, category, difficulty, created-by, date range), sortable table with thumbnail/icon, title, status pill, last updated, updated by.
- Create/Edit: a right-side drawer or full editor screen depending on content complexity (Roadmaps and Assessments use full-screen builders; Research Papers/Articles/Videos use a simpler drawer form).
- Every content type supports: Draft/Published/Archived status, Version History (same pattern as Section 7.5), Preview (renders exactly as students will see it), Restore from Archive, Bulk Import (CSV/JSON template per type) and Bulk Export.

## 8.3 Roadmaps

A roadmap is an ordered sequence of Days, each containing missions (reading, video, quiz, coding task). Builder UI is a drag-and-drop day list with an expandable editor per day.

- Fields: title, slug, description, cover image, target role (e.g. Frontend, AI/ML), difficulty, estimated duration (days), prerequisite roadmap (optional), status.
- Per-day fields: day number, title, objective, mission list (each mission: type, content ref, estimated minutes), unlock condition (sequential vs date-based).

### Roadmap — Supabase Tables & Endpoints

- `roadmaps`, `roadmap_days`, `roadmap_day_missions`, `roadmap_versions`.

| Method | Endpoint | Description |
|---|---|---|
| GET | /rest/v1/roadmaps | List roadmaps |
| POST | /functions/v1/roadmaps | Create roadmap with nested days/missions in one transaction |
| PATCH | /functions/v1/roadmaps/{id} | Update roadmap + reorder days |
| POST | /functions/v1/roadmaps/{id}/publish | Publish, versioned |

## 8.4 Assessments & Quizzes

- Assessment builder: question bank (MCQ, multi-select, coding, short-answer), sectioned by skill area, configurable time limit, passing score, randomize-question-order toggle, negative marking toggle.
- Quiz builder: lighter-weight version scoped to a single roadmap day, MCQ/true-false only, instant feedback toggle.

- Question bank is shared/reusable across assessments to avoid duplication; each question stores difficulty, skill tag, and correct-answer explanation.

#### Assessments/Quizzes — Supabase Tables

- `assessments`, `assessment_sections`, `questions` (shared bank), `assessment_questions` (join table with order/points), `quizzes`, `quiz_questions`.

## 8.5 Assignments

- Fields: title, instructions (rich text), roadmap day link (optional), submission type (file upload / GitHub link / text), due offset (days after unlock), rubric (list of criteria with point weights), auto-grading rule (optional, for code assignments via test-case execution).
- Grading queue view for graders separate from the builder, showing ungraded submissions first.

## 8.6 Projects

- Fields: title, brief (rich text), required tech stack tags, evaluation rubric, GitHub submission required (bool), AI-review enabled (bool — triggers Bedrock code-quality analysis on submission).

## 8.7 Research Papers, Articles, Videos

- Research Papers: title, authors, abstract, PDF upload or external DOI link, category, publish date.
- Articles: title, source, external URL or hosted rich-text body, category, curated-by.
- Videos: title, description, hosting (YouTube embed URL or uploaded MP4 via Storage + signed CDN URL), duration (auto-detected), category, transcript (optional, for accessibility).

## 8.8 Backend Services (Shared)

- Edge Function `content-crud` — generic, type-parameterized handler used by all eight content types for create/update/publish/archive, enforcing the shared status-machine (Draft → Published → Archived → Published) and writing to the matching `*_versions` table.
- Edge Function `bulk-import-content` — accepts a CSV/JSON upload, validates row-by-row against the type's schema, returns a per-row success/error report before committing (dry-run first, commit on confirm).
- Edge Function `bulk-export-content` — mirrors the student export pattern from Section 5.5.

## 8.9 Validation Rules

- A roadmap cannot be published with zero days, or a day with zero missions.
- An assessment cannot be published if any section has zero questions, or if passing score > total possible points.
- Bulk import rejects the entire batch only for structural errors (bad CSV headers); row-level content errors are reported individually and skippable.

## 8.10 Edge Cases

- Archiving a roadmap that students are currently assigned to → blocked with a warning listing affected student count; must reassign or explicitly force-archive (Super Admin only), which freezes those students' progress view but does not delete their history.
- Deleting a question from the shared bank that's used in a published assessment → soft-delete only (hidden from bank picker, retained for historical attempt scoring).

## 8.11 Future Scalability

- AI-assisted content generation (draft a roadmap day, generate quiz questions from a topic) via Bedrock, human-reviewed before publish.
- Content localization (multi-language roadmaps/articles).

# 9. Analytics

## 9.1 Purpose

A dedicated Analytics screen (Super Admin, Admin, and a content-scoped subset for Content Manager) providing deep, chart-driven insight beyond the dashboard's summary cards. Every chart supports a date-range picker (7d/30d/90d/custom) and CSV export of the underlying data.

## 9.2 Chart Inventory

| Chart | Type | Source |
|---|---|---|
| Student Growth | Line (cumulative + daily new) | dashboard_stats_daily |
| College Distribution | Horizontal bar, top 15 + 'Other' | students grouped by college |
| Role Distribution | Donut | students grouped by role_selected |
| Assessment Scores | Histogram (bucketed) | assessment_attempts |
| Talent Score Distribution | Histogram (bucketed) | students.talent_score |
| Completion Rates | Bar, by roadmap | roadmap_progress |
| Daily / Weekly / Monthly Active Users | Multi-line trend | student_activity_log |
| Learning Time | Stacked area, by roadmap category | student_activity_log durations |
| Roadmap Completion | Funnel (per-day drop-off) | roadmap_progress |
| Assignment Completion | Bar, on-time vs late vs missed | assignments submissions |
| Project Completion | Bar, submitted vs reviewed vs approved | projects |
| GitHub Connection % | Gauge / donut | students.github_connected |
| Top Colleges | Ranked table, by avg talent score | students |
| Top Skills | Bar, aggregated skill-graph scores | skill_graph |
| Top Performing Students | Ranked table, sortable | students, talent_score_history |

## 9.3 Layout

- Filter bar pinned at top: date range, college filter, role filter — cascades into every chart on the page.
- Responsive masonry-style grid, 2 charts per row on desktop (≥1280px), 1 per row on tablet/mobile.
- Each chart card: title, info tooltip explaining the metric, export icon (CSV of that chart's data), and a maximize icon (full-screen modal view).

## 9.4 Backend Services

- All charts read from pre-aggregated materialized views (`dashboard_stats_daily`, `college_distribution_view`, `skill_distribution_view`, etc.), refreshed hourly via `pg_cron`, so the Analytics page never runs expensive ad-hoc aggregations against raw tables at request time.
- Edge Function `get-analytics` accepts a chart key + filter payload and returns just that chart's data, allowing charts to load independently and in parallel rather than one large blocking payload.

## 9.5 API Endpoints

| Method | Endpoint | Description |
|---|---|---|
| GET | /functions/v1/get-analytics?chart=student_growth&range=30d | Single chart's data |
| GET | /functions/v1/get-analytics/export?chart=top_colleges | CSV export of one chart's underlying data |

## 9.6 Edge Cases

- Custom date range spanning before the platform's launch date → chart clamps to actual data availability and shows a note: 'Data available from {launch date}.'
- College/role filter combination yields zero data points → chart area shows an empty-state message instead of a broken/empty axis.

## 9.7 Future Scalability

- Cohort analysis (retention curves by signup week/college/role).
- Custom report builder allowing admins to compose their own charts from arbitrary dimensions/metrics.

# 10. Notifications

## 10.1 Purpose

A composer + delivery system for reaching students via in-app push and/or email, supporting immediate and scheduled sends. Available to Super Admin and Admin fully; Content Manager limited to New Blog notifications; Support Admin has no send access (view-only history).

## 10.2 Notification Types

- Announcement — general platform communication.
- Maintenance Notice — scheduled downtime alerts.
- Learning Reminder — nudges inactive students.
- Roadmap Update — informs affected students of content changes.
- New Blog — auto-triggered on publish (Section 7.4), but also composable manually for re-promotion.

## 10.3 Composer UI

- Fields: Type (select), Title, Message body (rich text for email, plain text for push with 120-char limit), Audience (All Students / Filtered segment reusing the Student Management filter engine / Specific students by search-select), Channels (in-app toggle, email toggle — at least one required), Send Time (Now / Schedule for later with date-time picker).
- Live preview pane showing both the in-app card render and the email render side-by-side.
- 'Send Test' button — delivers to the composing admin's own email only, for QA before a real send.

## 10.4 Delivery Pipeline

12. Admin submits the composer → a row is created in `notifications` with status `scheduled` (even for 'Now' sends, scheduled_at = now()).
13. A `pg_cron` job `dispatch-notifications` runs every minute, picks up due rows, and resolves the audience into a concrete recipient list (materialized into `notification_recipients` for idempotency and delivery tracking).
14. In-app channel: rows inserted into `student_notifications` (read by the student dashboard directly, real-time via Supabase Realtime).
15. Email channel: batched calls to the email provider (Resend/SES) via an Edge Function, respecting provider rate limits with exponential backoff on failures; delivery status (sent/bounced/failed) written back per recipient.

## 10.5 Backend Services

- Edge Function `send-notification` — validates payload, resolves 'Send Test', or creates the scheduled `notifications` row.
- `pg_cron` job `dispatch-notifications` as above.
- Edge Function `email-webhook-handler` — ingests provider delivery/bounce webhooks to update `notification_recipients.status`.

## 10.6 Supabase Tables

- `notifications` — id, type, title, body, audience_filter (jsonb), channels (text[]), status, scheduled_at, sent_at, created_by, created_at.
- `notification_recipients` — id, notification_id, student_id, channel, status (pending/sent/delivered/bounced/failed), sent_at.
- `student_notifications` — id, student_id, notification_id, title, body, is_read, created_at.

## 10.7 API Endpoints

| Method | Endpoint | Description |
|---|---|---|
| POST | /functions/v1/send-notification | Create/schedule a notification |
| POST | /functions/v1/notifications/test-send | Send-test to the current admin only |
| GET | /rest/v1/notifications?order=created_at.desc | Notification history list |
| GET | /rest/v1/notification_recipients?notification_id=eq.{id} | Per-recipient delivery status |
| DELETE | /rest/v1/notifications?id=eq.{id}&status=eq.scheduled | Cancel a not-yet-sent scheduled notification |

## 10.8 Validation Rules

- At least one channel and a non-empty audience resolution (>0 recipients) required before send/schedule is enabled.
- Push message body capped at 120 characters; email body has no hard cap but is warned above 2,000 characters.

## 10.9 Edge Cases

- Scheduled notification's audience filter would resolve to a different set at send time vs. compose time (e.g. 'Talent Score below 40') → resolved fresh at dispatch time by design, so it always reflects current data, and this behavior is called out in the UI with a note.
- Email bounces → student's email marked `email_bounced = true` after 2 consecutive bounces, surfaced as a warning badge on their profile, and excluded from future email sends until corrected.

## 10.10 Future Scalability

- SMS channel.
- A/B subject line testing for email campaigns.
- Per-student notification preference center (opt-out by category) surfaced from the student dashboard, respected here.

# 11. Settings

## 11.1 Overview

Settings is Super-Admin-only for most sub-sections (Admins, AI, API Keys, Supabase Settings), with a few sub-sections available to Admin (Email Templates, Talent/Assessment/Roadmap Rules).

## 11.2 Admins

- Table of all staff accounts: name, email, role, status, last login, created by/at.
- 'Invite Admin' flow: Super Admin enters name, email, role → `provision-admin` Edge Function creates the `auth.users` + `admin_profiles` row with a temporary password and sends an invite email with a forced password-reset link.
- Deactivate/reactivate admin; change role (subject to the RBAC rules in Section 3, including the 'last Super Admin' protection).

## 11.3 Roles & Permissions

Read-only view of the built-in role matrix (Section 3.2) plus, for future custom roles, a permission-editing UI bound to the `permissions` JSONB schema.

## 11.4 Email Templates

- Editable templates: Welcome/Onboarding, Password Reset, Weekly Report, Notification (Announcement/Reminder/Maintenance), Blog Published.
- Template editor supports merge variables (e.g. `{{student_name}}`, `{{roadmap_title}}`) with a live preview using sample data.

## 11.5 Talent Score, Assessment, Roadmap Rules

- Talent Score Rules: configurable weightings per input signal (assessment performance, project quality, GitHub activity, roadmap pace) that feed the scoring Edge Function; changes apply prospectively and trigger a recompute job.
- Assessment Rules: global defaults (passing threshold, retake cooldown period, max retakes) overridable per-assessment.
- Roadmap Rules: default unlock behavior (sequential vs date-based), grace-period days before a student is flagged inactive.

## 11.6 AI Settings — Amazon Bedrock Configuration

- Model selection per use case (talent scoring, weekly report generation, project code review, dropout prediction) — dropdown of available Bedrock model IDs.
- Prompt template editor per use case, versioned (same version-history pattern as content).
- Token/cost budget guardrails: monthly spend cap with alerting at 80%/100% via the Notification system (to admins, not students).
- AWS credentials are never entered directly in the UI — configured via environment secrets on the Supabase Edge Function runtime; this screen only manages model/prompt configuration and displays masked key status ('Connected' / 'Not configured').

## 11.7 API Keys

- Table of service integrations (Bedrock, GitHub App, Email provider, Storage CDN) showing connection status and last-verified timestamp, with a 'Test Connection' action per row.
- Actual secret values are write-only (entered once, never displayed again) and stored in Supabase Vault, not in a plain column.

## 11.8 Supabase Settings & Storage

- Display of current project region, connection pooling status (for admin awareness, not editable here).
- Storage bucket usage dashboard: size per bucket (blog-images, avatars, exports, videos, research-papers), with a cleanup action for expired export files.

## 11.9 Branding

- Logo upload (used across the Admin Portal shell), primary/accent color override (defaults to CalibiAI green), favicon.

## 11.10 Backend Services

- Edge Function `provision-admin` (Section 2.7).
- Edge Function `recompute-talent-scores` — triggered on Talent Score Rule change, batches through all active students.
- Edge Function `test-integration-connection` — pings the given service (Bedrock, GitHub App, email provider) and returns latency + status.

## 11.11 Supabase Tables

- `email_templates`, `talent_score_rules`, `assessment_rules`, `roadmap_rules`, `ai_model_configs`, `ai_prompt_templates` (versioned), `integration_status`, `branding_settings`.

## 11.12 Edge Cases

- Changing the active AI model mid-recompute job → the in-flight job finishes on the model it started with; the new model applies to the next run, to avoid mixed-model inconsistency within one batch.
- Deactivating an admin who is the sole author of unpublished draft content → content remains editable by other Admin+ roles; ownership is not deleted, just the account.

## 11.13 Future Scalability

- Full audit-log export for compliance (SOC2-style).
- Multi-environment (staging/production) settings profiles.

# 12. Database Design

## 12.1 Overview

All tables live in Supabase Postgres. Every table has Row Level Security enabled by default; policies are additive per role. This section consolidates the full schema referenced throughout this document, plus buckets, triggers, and indexing strategy.

## 12.2 Full Table Inventory

| Table | Domain | Key Relationships |
|---|---|---|
| admin_profiles | Auth/RBAC | FK auth.users; FK admin_roles |
| admin_roles | Auth/RBAC | Referenced by admin_profiles.role |
| admin_login_logs | Auth | FK admin_profiles (nullable) |
| admin_permission_audit | Auth/RBAC | FK admin_profiles (target + actor) |
| admin_activity_logs | Audit | FK admin_profiles; polymorphic target |
| students | Students | FK auth.users; FK roadmaps (current) |
| student_login_logs | Students | FK students |
| student_activity_log | Students | FK students |
| export_jobs | Students | FK admin_profiles |
| assessment_attempts, quiz_attempts | Learning | FK students; FK assessments/quizzes |
| assignments, assignment_submissions | Learning | FK roadmap_days; FK students |
| projects, project_submissions | Learning | FK students |
| roadmap_progress, daily_missions | Learning | FK students; FK roadmap_days |
| github_analysis | Learning | FK students |
| talent_score_history, skill_graph | Learning | FK students |
| weekly_reports | Learning | FK students |
| roadmaps, roadmap_days, roadmap_day_missions, roadmap_versions | Content | self-referencing prerequisite |
| assessments, assessment_sections, questions, assessment_questions | Content | shared question bank |
| quizzes, quiz_questions | Content | FK roadmap_days |
| research_papers, articles, videos | Content | flat, category-tagged |
| blog_posts, blog_post_versions, blog_categories | Blog | FK admin_profiles (author) |
| notifications, notification_recipients, student_notifications | Notifications | FK students; FK admin_profiles |

| Table | Domain | Key Relationships |
|---|---|---|
| email_templates, talent_score_rules, assessment_rules, roadmap_rules | Settings | — |
| ai_model_configs, ai_prompt_templates, integration_status, branding_settings | Settings | — |
| dashboard_stats_daily, college_distribution_view, skill_distribution_view | Analytics (materialized views) | aggregated |
| system_error_logs | Ops | — |

## 12.3 Representative Schema — Core Tables

```
create table admin_profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  full_name text not null,
  email text not null unique,
  role text not null references admin_roles(role),
  permissions jsonb not null default '{}',
  is_active boolean not null default true,
  avatar_url text,
  created_by uuid references admin_profiles(id),
  last_login_at timestamptz,
  created_at timestamptz not null default now()
);

create table students (
  id uuid primary key default gen_random_uuid(),
  auth_user_id uuid references auth.users(id) on delete cascade,
  full_name text not null,
  email text not null unique,
  phone text,
  college text, branch text, graduation_year int,
  role_selected text,
  avatar_url text,
  github_username text, github_connected boolean default false,
  account_status text not null default 'active'
    check (account_status in ('active','inactive','suspended')),
  current_roadmap_id uuid references roadmaps(id),
  current_day int default 0,
  progress_percent numeric(5,2) default 0,
  assessment_score numeric(5,2),
  talent_score numeric(6,2),
  email_bounced boolean default false,
  created_at timestamptz not null default now(),
  last_login_at timestamptz
);

alter table students enable row level security;
create index idx_students_search on students
```

```
  using gin ((full_name || ' ' || email || ' ' || coalesce(phone,'')) gin_trgm_ops);
create index idx_students_filters on students
  (college, branch, account_status, last_login_at);
```

## 12.4 Triggers

- `trg_blog_publish_notify` (AFTER UPDATE on blog_posts, WHEN status changes to Published) → invokes the notification fan-out Edge Function via `pg_net`.
- `trg_student_login_touch` (AFTER INSERT on student_login_logs) → updates `students.last_login_at`.
- `trg_admin_permission_audit` (AFTER UPDATE on admin_profiles, WHEN role or permissions changes) → inserts a row into `admin_permission_audit`.
- `trg_version_snapshot` — generic trigger function attached to blog_posts, roadmaps, assessments, etc. on explicit-save events → inserts into the matching `*_versions` table.

## 12.5 Storage Buckets

| Bucket | Access | Contents |
|---|---|---|
| avatars | Public read, owner write | Student & admin profile photos |
| blog-images | Public read, admin write | Cover/featured/inline blog images |
| research-papers | Authenticated read, admin write | PDF uploads |
| videos | Authenticated read, admin write | Uploaded MP4 content |
| exports | Private, signed URL only | Generated CSV/Excel/PDF exports |
| submissions | Private, owner + admin read | Assignment/project file submissions |

## 12.6 Row Level Security Summary

Every table follows the pattern established in Section 3.4: SELECT policies keyed off `auth.jwt() ->> 'role'`, with UPDATE/DELETE policies further restricted per the role matrix. Tables containing student PII (students, weekly_reports, github_analysis) explicitly exclude `content_manager` from SELECT policies. All policies are defined in versioned SQL migration files, never edited ad-hoc in the Supabase dashboard, to keep environments reproducible.

# 13. API Design

## 13.1 Conventions

- Two API surfaces are used throughout this document: Supabase's auto-generated PostgREST layer (`/rest/v1/...`) for straightforward CRUD covered by RLS, and custom Supabase Edge Functions (`/functions/v1/...`) for anything involving business logic, multi-table transactions, external calls (Bedrock, email), or role-permission checks beyond RLS.
- All requests require an `Authorization: Bearer <access_token>` header; PostgREST additionally requires the `apikey` header per Supabase convention.
- All responses are JSON. Successful responses use standard 2xx codes; list endpoints return `{ data: [...], count: number }`.
- All Edge Functions validate input with a schema library (Zod) before touching the database, returning 422 with a field-level error map on failure.

## 13.2 Standard Error Envelope

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "One or more fields are invalid.",
    "fields": { "email": "Must be a valid email address" }
  }
}
```

## 13.3 Standard Error Codes

| HTTP Status | Code | Meaning |
|---|---|---|
| 400 | BAD_REQUEST | Malformed request body/params |
| 401 | UNAUTHENTICATED | Missing or expired session |
| 403 | FORBIDDEN | Authenticated but not authorized for this action (role/permission) |
| 404 | NOT_FOUND | Resource does not exist or is not visible under RLS |
| 409 | CONFLICT | Version mismatch / concurrent edit / business-rule conflict |
| 422 | VALIDATION_ERROR | Field-level validation failure |
| 429 | RATE_LIMITED | Too many requests (login attempts, exports, etc.) |
| 500 | INTERNAL_ERROR | Unexpected server error — logged to system_error_logs |

## 13.4 Authentication & Authorization Flow (all Edge Functions)

16. Verify the Supabase JWT from the Authorization header.

17. Load the caller's `admin_profiles` row (role + permissions + is_active) — never trust the JWT's role claim alone for write operations.
18. Check the specific permission required for the action; return 403 with a descriptive message if absent.
19. Execute business logic inside a transaction where multiple tables are touched.
20. Write an `admin_activity_logs` row for any mutating action (who, what, before/after where applicable).

## 13.5 Consolidated Endpoint Reference

The full endpoint set is documented inline within each module's chapter (Sections 2-11). The table below summarizes endpoint groupings by base path for quick orientation.

| Base Path | Module | Chapter |
|---|---|---|
| /auth/v1/* | Authentication | 2 |
| /functions/v1/admin/*, /rest/v1/admin_roles | RBAC | 3 |
| /functions/v1/get-dashboard-summary, /rest/v1/admin_activity_feed | Dashboard | 4 |
| /functions/v1/list-students, /export-students, /bulk-student-action | Student Management | 5 |
| /functions/v1/get-student-profile, /students/{id}/* | Student Profile | 6 |
| /rest/v1/blog_posts, /functions/v1/publish-blog-post | Blog | 7 |
| /rest/v1/roadmaps, /assessments, /quizzes, ... /functions/v1/content-crud | Content Management | 8 |
| /functions/v1/get-analytics | Analytics | 9 |
| /functions/v1/send-notification, /rest/v1/notifications | Notifications | 10 |
| /functions/v1/provision-admin, /rest/v1/email_templates, ai_model_configs | Settings | 11 |

# 14. UI Design

## 14.1 Design Principles

- Enterprise-grade minimalism inspired by Vercel, Linear, Notion, Supabase, and Stripe: generous whitespace, restrained color use, information density achieved through layout rather than clutter.
- White (#FFFFFF) surfaces with CalibiAI green (#12B76A primary, #027A48 dark, #ECFDF3 light tint) as the sole accent — used for primary actions, active states, and data-positive indicators only, never decoratively.
- Typography: Inter (or system-ui fallback) throughout; a monospace face (JetBrains Mono) for code, IDs, and tabular numerals where alignment matters.

## 14.2 Design Tokens

| Token | Value | Usage |
|---|---|---|
| --color-bg | #FFFFFF | Page background |
| --color-surface | #F9FAFB | Cards, panels |
| --color-border | #D0D5DD | Dividers, table borders |
| --color-text-primary | #101828 | Headings, body |
| --color-text-secondary | #475467 | Labels, meta text |
| --color-primary | #12B76A | Primary buttons, active nav, links |
| --color-primary-dark | #027A48 | Hover states, headings accent |
| --color-primary-tint | #ECFDF3 | Success backgrounds, selected rows |
| --color-danger | #D92D20 | Destructive actions, error states |
| --color-warning | #F79009 | Caution badges |
| --radius-sm / md / lg | 6px / 8px / 12px | Buttons/inputs / cards / modals |
| --shadow-card | 0 1px 2px rgba(16,24,40,0.05) | Default card elevation |
| --font-body | 14px / 1.5 | Default text size/line-height |

## 14.3 Core Components

### Buttons

- Primary: solid green background, white text, 8px radius, 40px height for standard, 36px for compact-in-table.
- Secondary: white background, grey border, dark text.
- Destructive: white background, red border/text default, solid red on hover/confirm step.
- All buttons: disabled state at 40% opacity + not-allowed cursor; loading state replaces label with a spinner, width does not collapse.

### Cards

White surface, 1px #D0D5DD border, 12px radius, `--shadow-card`, 20px internal padding. Stat cards use a top-aligned label/value pair with a right-aligned trend badge.

### Tables

- Sticky header row, 44px row height, zebra-free (uses hover highlight instead: `--color-surface` on row hover), right-aligned numeric columns with tabular figures.
- Status/score values render as pill badges, never plain colored text alone (color + icon/label for accessibility).

### Filters & Search

Search inputs use a leading magnifying-glass icon and a trailing clear (x) icon when populated. Filter drawers slide in from the right, 400px wide, with a sticky 'Apply' / 'Reset' footer.

### Empty States

Centered icon or simple line illustration (no stock photography), a one-line explanation, and — where actionable — a single primary button (e.g. 'Clear filters', 'Create your first roadmap').

### Loading States

Skeleton screens matching final layout dimensions are used everywhere over spinners, except for button-level and small inline async actions, which use a small spinner in place of the icon/label.

### Pagination

Numbered pagination with prev/next arrows for tables under ~50 pages of results; 'Load more' infinite scroll used only for activity/timeline feeds, never for primary data tables (to keep exports/selection state predictable).

## 14.4 Dark Mode

Dark mode is a token swap, not a redesign: surfaces invert to near-black (#0C0F14 background, #12151B cards), text inverts to off-white, borders shift to #22262E, and the CalibiAI green accent is preserved as-is (tested for AA contrast on the dark background). Theme preference stored per-admin (`admin_profiles.theme_preference`) and defaults to system preference.

## 14.5 Responsive Design

- Breakpoints: mobile <640px, tablet 640-1024px, desktop >1024px.
- Sidebar collapses to a bottom-accessible slide-over on mobile; top bar becomes a single row with a hamburger trigger.
- Data tables on mobile convert to a stacked card-per-row layout (label/value pairs) rather than horizontal scroll, for the primary Student Management and Content tables; secondary/dense tables (e.g. version history) retain horizontal scroll with a visible scroll-shadow affordance.
- Charts resize fluidly and reduce to essential series on narrow viewports (e.g. legend moves below chart, tick labels abbreviate).

## 14.6 Accessibility & Keyboard Navigation

- WCAG 2.1 AA target: minimum 4.5:1 text contrast, visible focus rings (2px green outline, 2px offset) on every interactive element.
- Full keyboard operability: Tab/Shift+Tab order follows visual layout; Cmd/Ctrl+K opens the command palette from anywhere; Esc closes any drawer/modal; Enter/Space activates focused controls.
- All icons-only buttons have `aria-label`; all form fields have associated `<label>` elements; status pills include text, not color alone.

- Tables use proper `<th scope>` semantics; live regions (`aria-live="polite"`) announce toast notifications and async save states to screen readers.

# 15. Security

## 15.1 Authentication

Handled entirely by Supabase Auth (email/password). No public signup route exists in the Admin Portal codebase — the only account-creation path is the Super-Admin-gated `provision-admin` Edge Function, which uses the Supabase service role key (server-side only, never exposed to the client).

## 15.2 Authorization

Defense in depth across three layers: (1) UI-level hiding of unauthorized actions/navigation for UX only, (2) Edge Function-level role/permission checks against the live `admin_profiles` row before executing any mutation, (3) Postgres Row Level Security as the ultimate enforcement layer that holds even if application code has a bug.

## 15.3 Row Level Security

- RLS is enabled on every table with no exceptions; there is no table relying solely on application-layer checks.
- Policies are written and reviewed as SQL migrations, checked into version control, never edited directly via the Supabase dashboard in production.
- PII-bearing tables (students, weekly_reports, github_analysis, student_activity_log) explicitly deny SELECT to `content_manager`.

## 15.4 Audit Logs

- `admin_activity_logs` — every mutating action by any admin (student edits, content publishes, bulk actions, role changes, deletions) with actor, action type, target record, before/after diff where relevant, IP, timestamp. Retained indefinitely, exportable by Super Admin.
- `admin_permission_audit` — dedicated log for role/permission changes specifically, surfaced on each admin's Settings profile.

## 15.5 Failed Login Tracking & Rate Limiting

- `admin_login_logs` records every attempt, success or failure, with IP and user agent.
- 5 failed attempts for the same email within 15 minutes → account temporarily locked for 15 minutes; the lock is visible to Super Admins in Settings → Admins with a manual 'Unlock now' action.
- Password-reset requests rate-limited to 3 per email per hour to prevent email-bombing.
- Export and bulk-action endpoints are rate-limited per-admin (e.g. 10 export jobs / 10 minutes) to prevent resource exhaustion.

## 15.6 Session Expiry

Access tokens expire hourly with automatic refresh while the 'Remember me' session is active; idle timeout at 30 minutes; absolute session ceiling at 12 hours regardless of activity, per Section 2.5.

## 15.7 Encryption

- All traffic over TLS 1.2+ (enforced by Vercel and Supabase infrastructure).
- Data at rest encrypted by Supabase's underlying Postgres/storage encryption.

- Third-party secrets (Bedrock credentials, email provider API keys, GitHub App secrets) stored in Supabase Vault / Edge Function environment secrets, never in application tables, never returned by any API response after initial entry.
- Signed, time-limited URLs (24h expiry) for all export files and private submission uploads rather than public bucket access.

## 15.8 Additional Hardening

- Content Security Policy restricting script sources; Supabase and CDN domains explicitly allow-listed.
- CSRF is largely mitigated by the bearer-token (not cookie-session) auth model; state-changing GETs are avoided entirely.
- Dependency and container image scanning in CI before deploy.

# 16. Future Features

## 16.1 Overview

The following are intentionally out of scope for the initial implementation-ready release but the data model and module boundaries above have been designed so none of them require a structural rewrite to add later.

## 16.2 Roadmap of Future Capabilities

| Feature | Depends On | Notes |
|---|---|---|
| Bulk Email (campaign-style, beyond transactional) | Notifications module (Sec. 10) | Adds campaign scheduling, open/click tracking |
| College Management | New `colleges` reference table | De-duplicates free-text college entries into a managed directory with placement stats |
| Certificate Generator | Roadmap completion data (Sec. 8.3) | PDF generation on roadmap completion, verifiable via a public certificate ID lookup |
| Placement Dashboard | Student profile + College Management | Tracks student placement outcomes linked to performance data |
| Recruiter Management | Placement Dashboard, new RBAC role | External-facing recruiter accounts, a new role class distinct from internal admin roles |
| AI Analytics | Analytics module (Sec. 9) + Bedrock | Natural-language querying over analytics data |
| Predict Student Dropout | Talent Score + Activity Log + Bedrock | Risk-scoring model surfaced on Student Profile and as a Support Admin worklist |
| AI Student Recommendations | Skill Graph + Bedrock | Personalized next-roadmap / resource suggestions surfaced to admins for review before being shown to students |

## 16.3 Design Guardrails for Future Work

- New roles (e.g. Recruiter) should extend the existing `admin_roles` table rather than branching into a parallel auth system.
- New AI-driven features route through the same `ai_model_configs` / `ai_prompt_templates` settings infrastructure (Section 11.6) rather than hardcoding model calls in feature code.
- Any new student-data-bearing feature must be reviewed against the PII-access rules in Section 3.2 and 15.3 before implementation.